Software Engineering

DR. SAMAR A. SAID

Points to be covered:

What Activity Diagram

Why Activity Diagrams .. ?

Activity Diagrams Notations

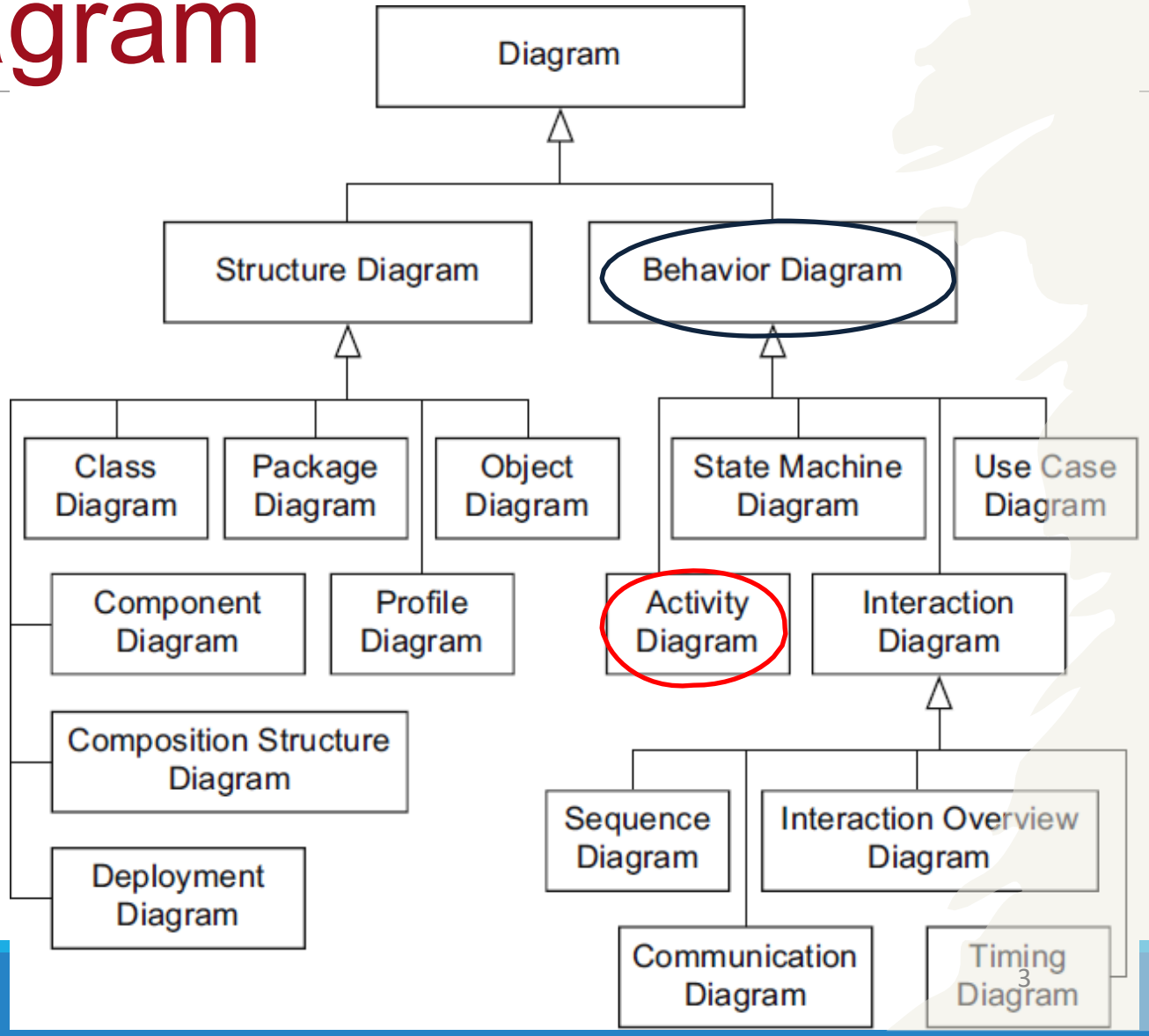
New notations

- Control nodes
- Object nodes
- Pre & post condition
- SendSignalAction
- Time triggers and Time events
- AcceptEventAction
- InterruptibleActivityRegion
- Exception
- Expansion Region

Behavioral [Dynamic] Modelling

the Activity Diagram

An Activity Diagram is a type of UML (Unified Modeling Language) diagram used to visualize the flow of activities or actions in a system or process. It shows step-by-step flow of a process from start to finish. Useful for understanding how a system works internally.



Why use Activity Diagrams

1. Models Workflows and Business Processes

- Helps in understanding and documenting how a business process or system works.
- Used in **business process modeling** to represent interactions between users, systems, and external entities.

Example: A bank's loan approval process can be represented using an activity diagram, showing the steps from application submission to loan approval or rejection.

2. Represents System Behavior

- Illustrates how different components of a system interact with each other.
- Shows the **flow of control** between various activities.

Example: An online shopping system can have an activity diagram showing steps like "Browse Products → Add to Cart → Payment → Order Confirmation."

Why use Activity Diagrams

3. Improves System Understanding

- Provides a **clear, visual representation** of complex processes.
- Helps developers, analysts, and stakeholders **understand the flow** without needing deep technical knowledge.

Example: Non-technical stakeholders can easily grasp the process of user registration, login, and authentication through an activity diagram.

4. Supports Parallel Processing

- Represents **concurrent activities** using fork and join nodes.
- Shows how different tasks can execute **simultaneously**.

Activity Diagram [the Activities]

An activity is a directed graph:

- **Nodes:** actions, start, end, or data stores
- **Edges:** for control and data flow.

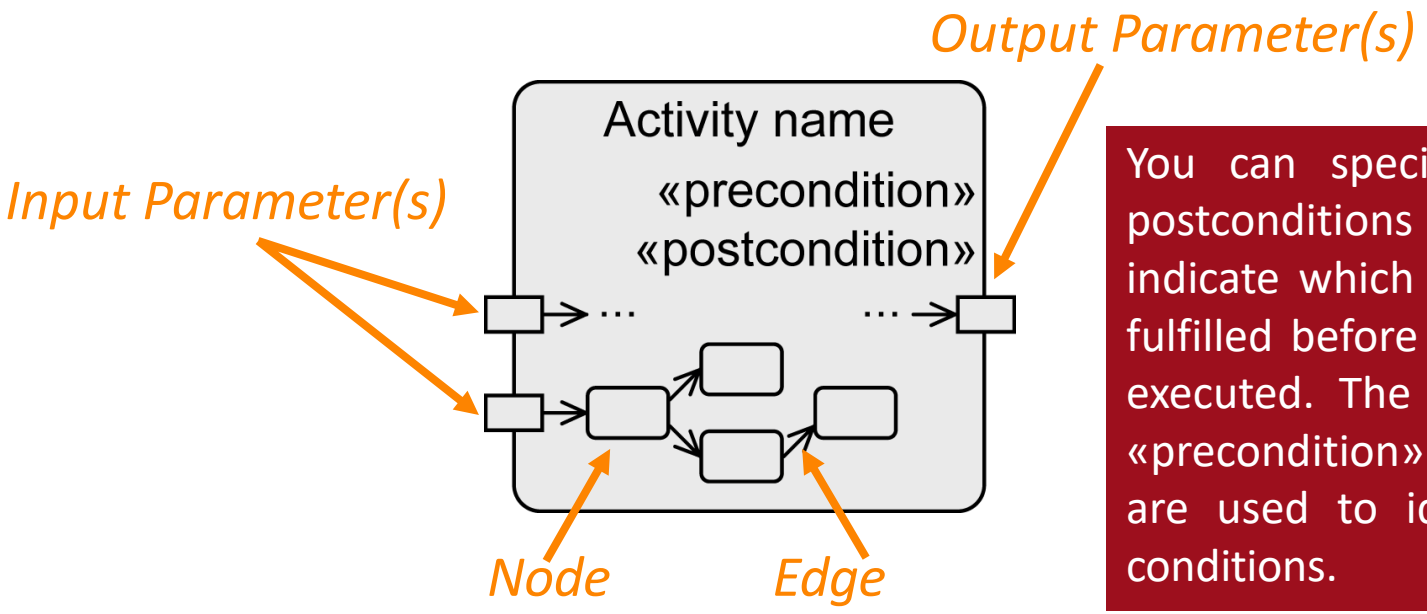
Control flow and object flow define the execution.

Optional:

- Parameters (input and output),
- Pre-conditions and Post-conditions.

Activity Diagram

An activity, just like an operation, may have parameters. These are shown as rectangles arranged overlapping at the boundary of the activity.

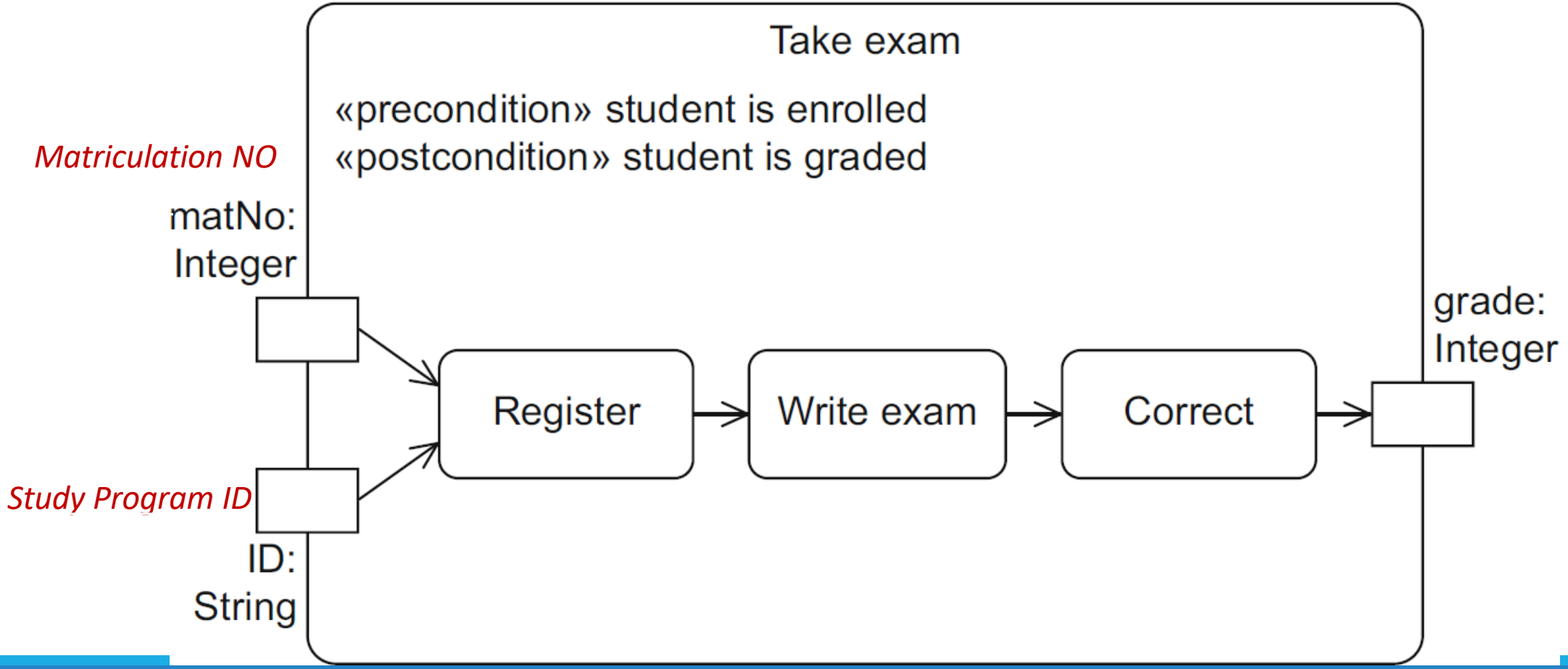


You can specify preconditions and postconditions for an activity. These indicate which conditions have to be fulfilled before or after the activity is executed. The keywords/stereotypes «precondition» and «postcondition» are used to identify the respective conditions.

To make the diagram easier to read, you should position **input parameters** *at the left or upper boundary* and **output parameters** *at the right or lower boundary* of the activity. This allows an activity to be read from left to right and/or from top to bottom.

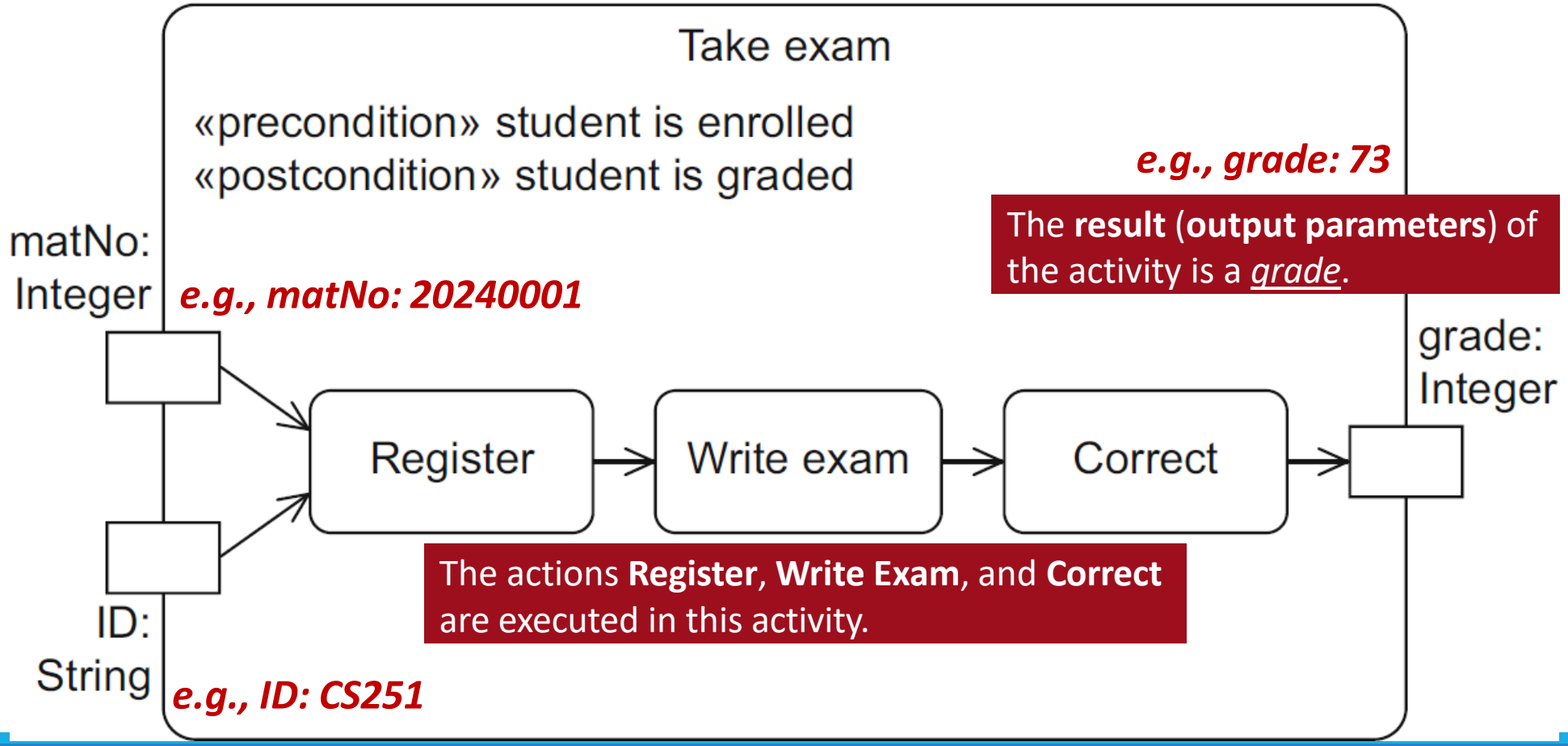
Activity Diagram [the Activities]

This activity diagram shows the steps (i.e., actions) necessary to execute the activity **Take Exam**.



Activity Diagram [the Activities]

The **input parameters** are the *Matriculation Number (Student ID)* and the *Study Program ID* of a student.

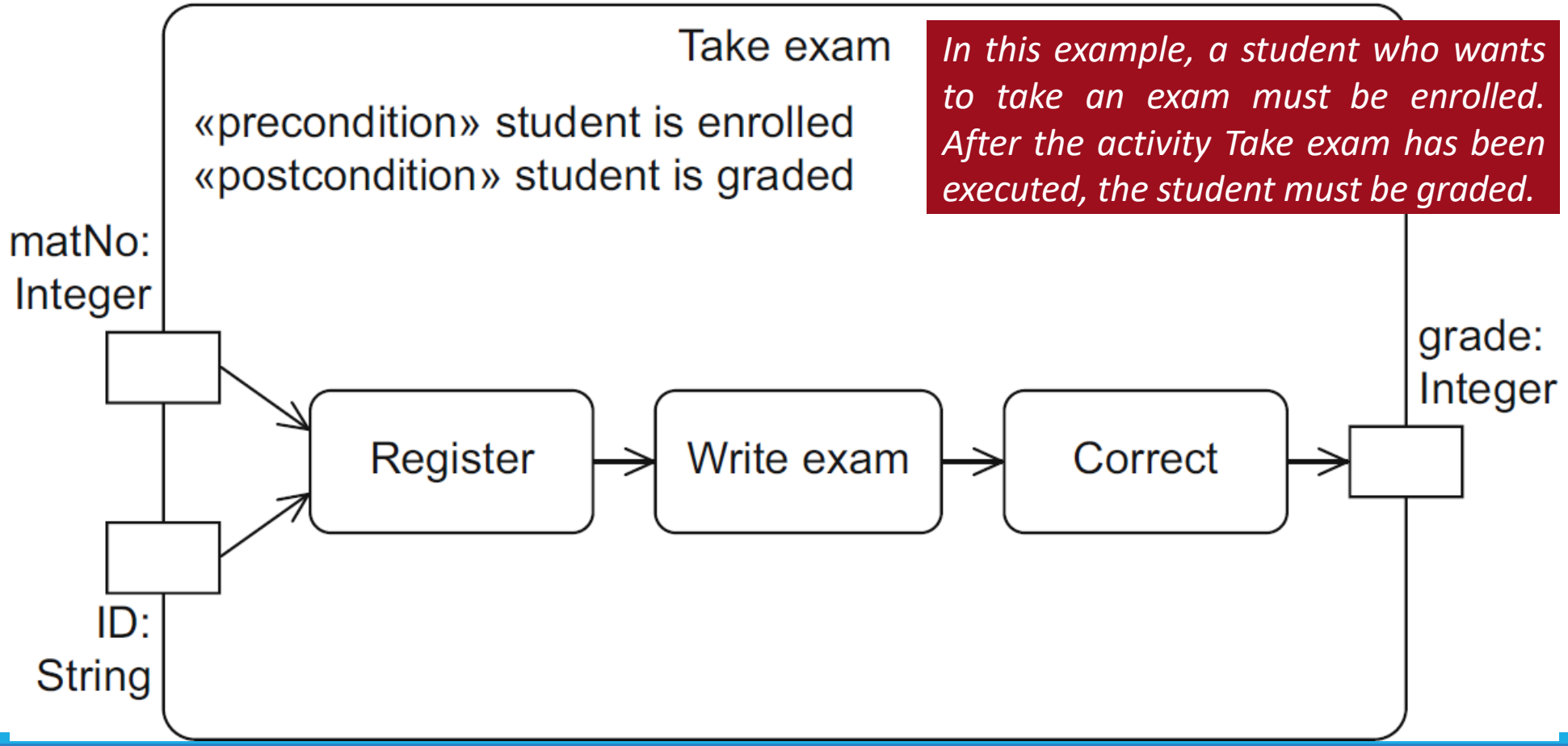


The **result** (**output parameters**) of the activity is a *grade*.

The actions **Register**, **Write Exam**, and **Correct** are executed in this activity.

This example diagram does not show, however, who performs which action.

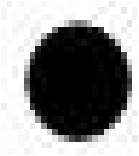
Activity Diagram [the Activities]



In this example, a student who wants to take an exam must be enrolled. After the activity Take exam has been executed, the student must be graded.

Activity Diagrams Notations

- **Initial node**:
  - The filled in circle is the starting point of the diagram.
  - **Symbol:**



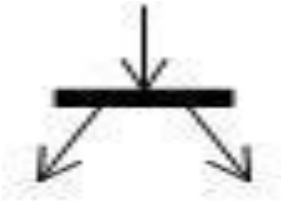
- **Activity final node**:
  - The filled circle with a border is the ending point.
  - **Symbol**:

Activity Diagrams Notations

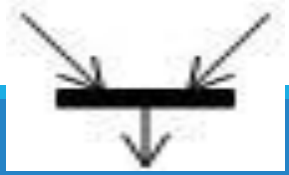
- **Activity (Action)**
  - The rounded rectangles represent activities that occur. An activity may be physical, such as *Inspect Forms*, or electronic, such as *Display Create Student Screen*.

Activity Diagrams Notations

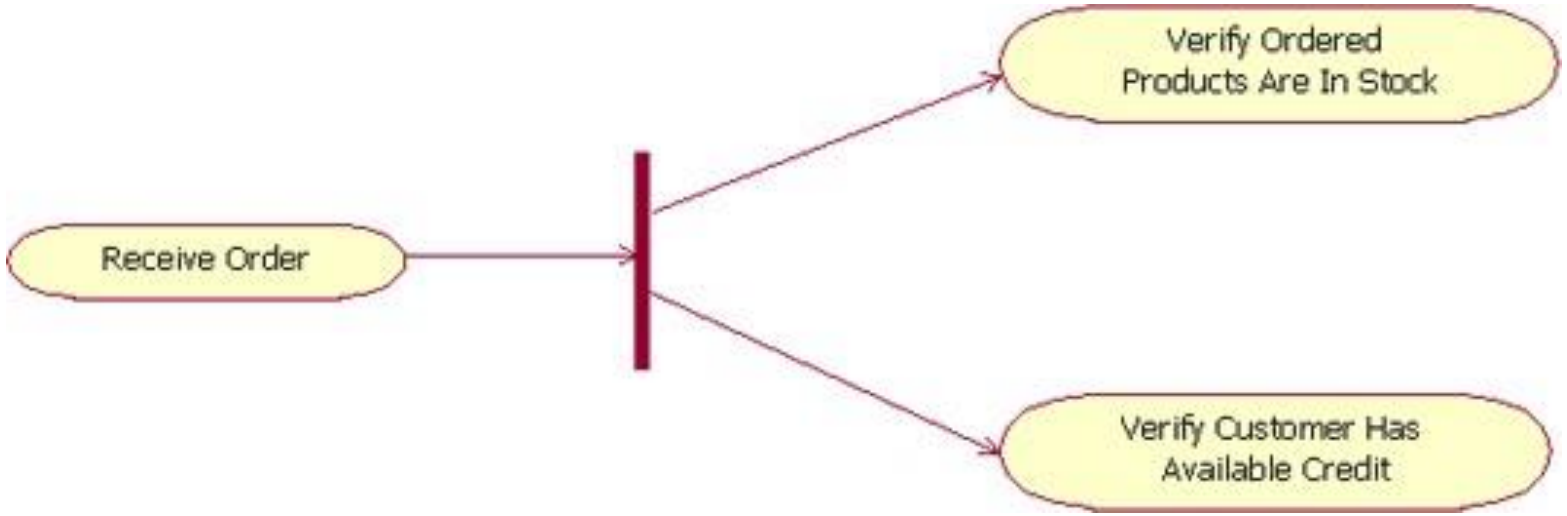
- **Parallel Activities:**
- **Fork**
  - A black bar with one flow going into it and several leaving it. This denotes the beginning of parallel activity. When the incoming transition is triggered, all of the outgoing transitions are taken in parallel.
  - Symbol:



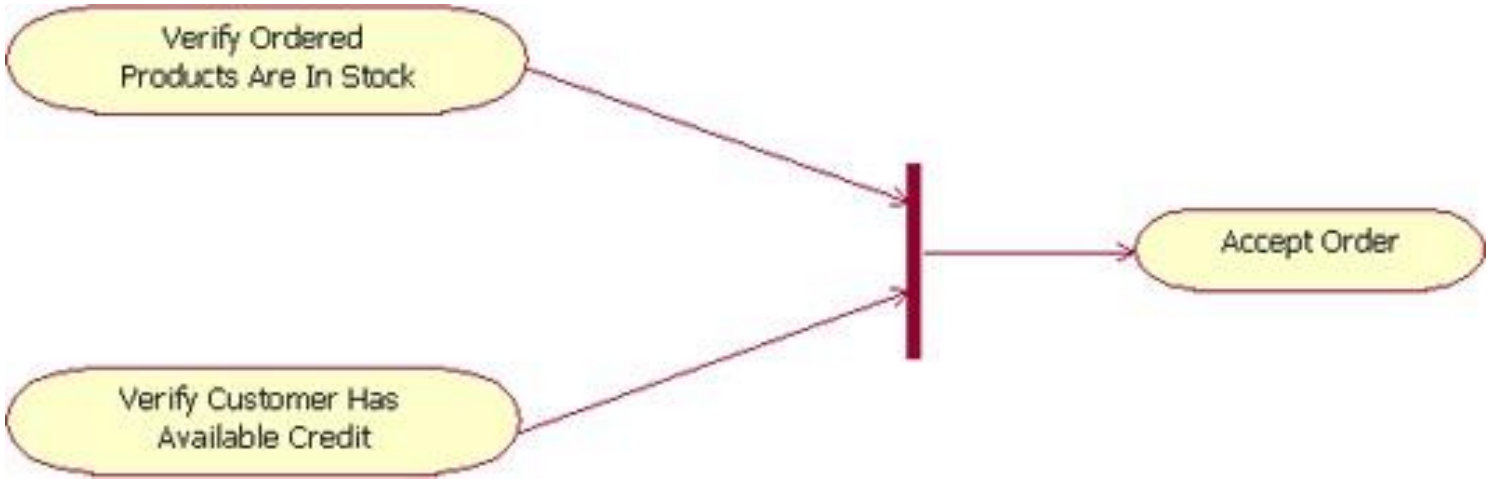
- **Join**
  - A black bar with several flows entering it and one leaving it. This denotes the end of parallel processing.
  - Symbol:



Fork example

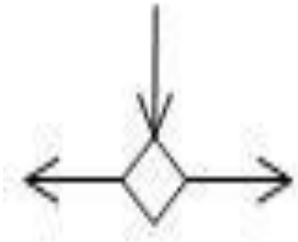


Join Example

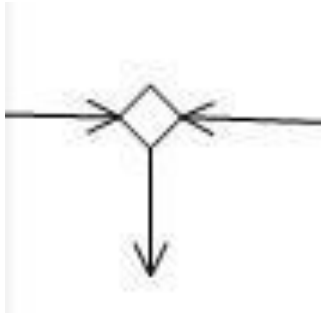


Activity Diagrams Notations

- **Branch (Decision node):**
  - Also called decision. a diamond with one flow entering and several leaving. The flows leaving include conditions although some modelers will not indicate the conditions if it is obvious.



- **Guard Condition**:
  - Text such as *[Incorrect Form]* on a flow, defining a guard which must evaluate to true in order to traverse the node.
  - This node is used to represent the actual test condition that is used to determine which of the paths exiting the decision node is to be traversed.



Merge:

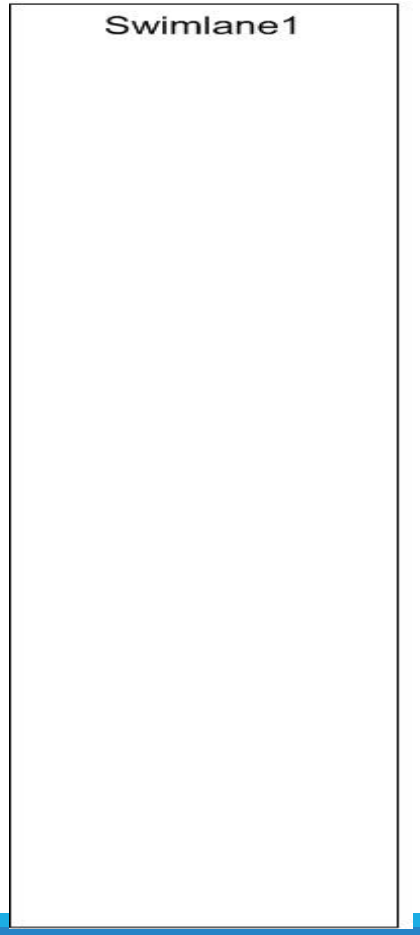
- A diamond with several flows entering and one leaving. The implication is that all incoming flows must reach this point until processing continues.

Key Differences

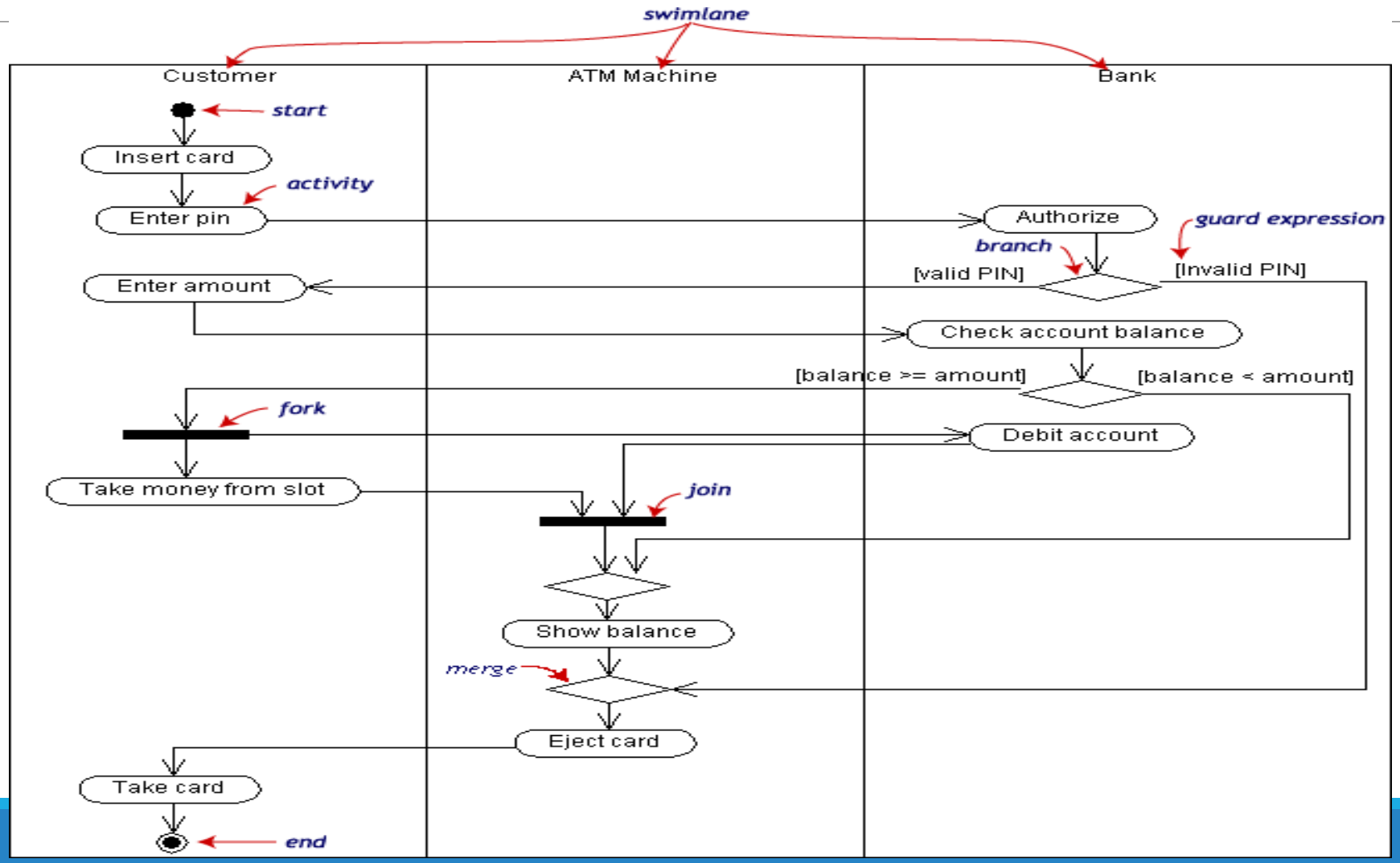
Feature	Merge	Join
Type of paths	Alternative (if/else)	Parallel (simultaneous)
Execution	Only one path is active	All paths run together
Waiting	No waiting	Waits for all
Symbol	Diamond	Thick bar
Used after	Decision	Fork

Activity Diagrams Notations

- **Partition**.
  - Also called swimlanes, indicating who/what is performing the activities
  - Are used to group actions according to the organization or class responsible for those actions.
  - Are divided by solid vertical lines.
  - Can be used in the context of concurrency.
  - An activity can only belong to one partition, but transitions between actions may cross partitions.



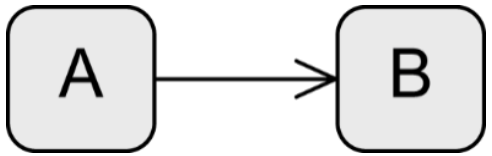
Swimlane example



Activity Diagram [the Edges]

Control–flow **edges vs.** Object–flow **edges** | Guards

Edges connect activities and actions to one another and express the execution order ..

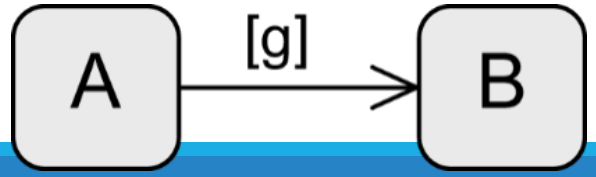


Types of Edges:

- **Control flow edges;** Define the order between nodes.
- **Object flow edges;** Used to exchange data or objects *(thus express a data/causal dependency between nodes)*.

Guard (condition):

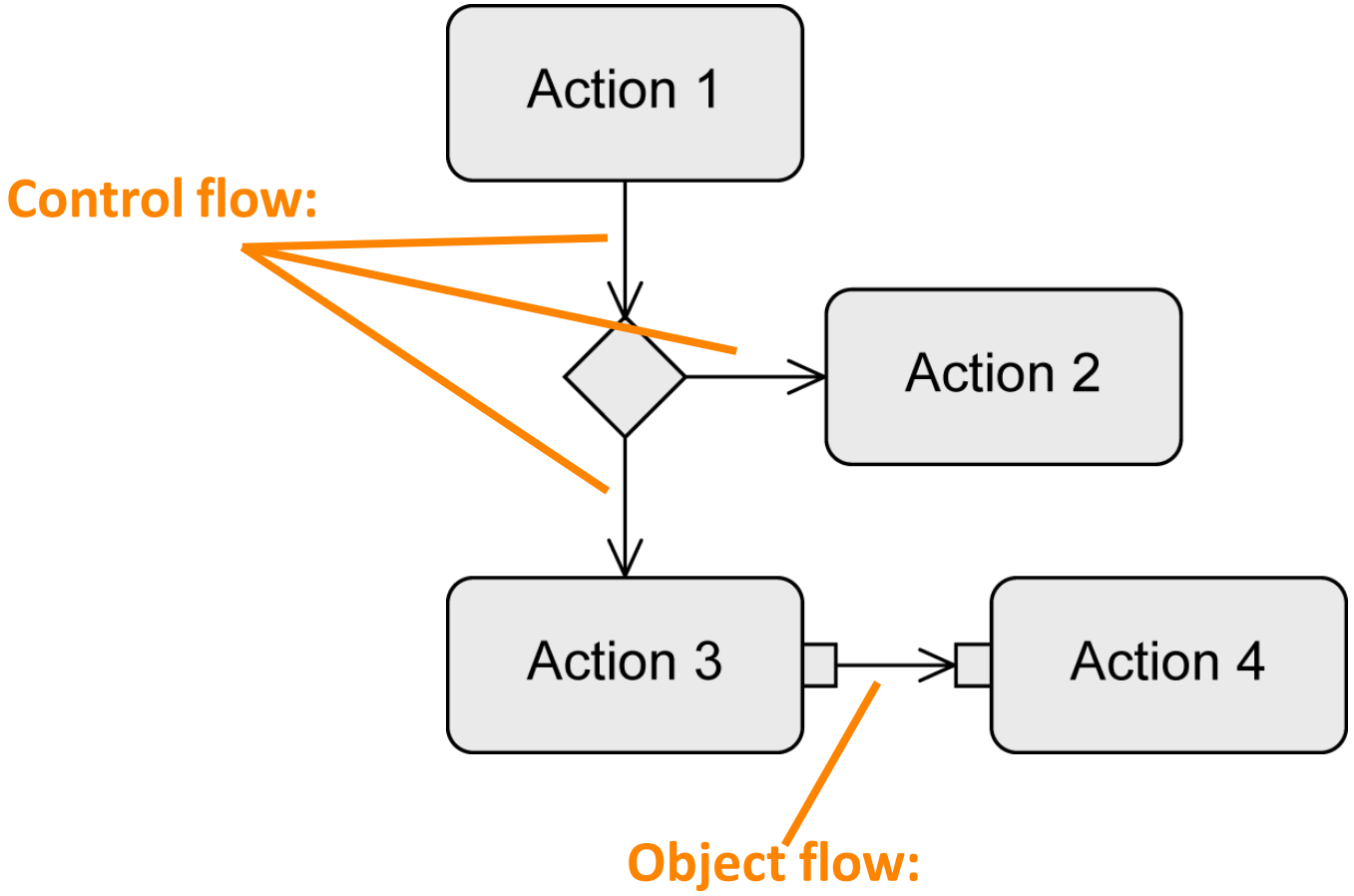
- Control and object flow only continue if guards in square brackets evaluate to true.



an **Example**

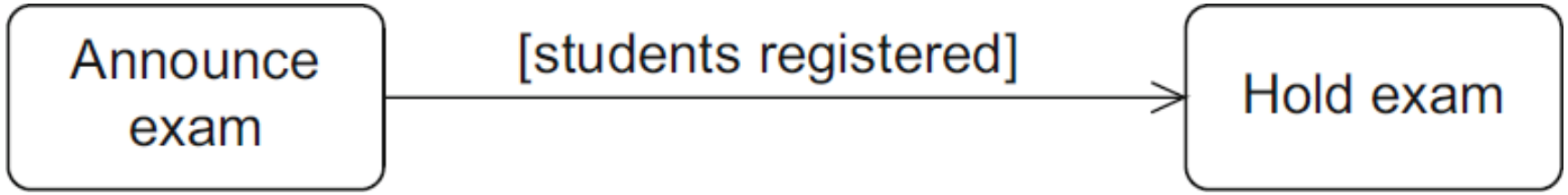
Activity Diagram [the Edges]

Control–flow edges vs. Object–flow edges | Guards



Activity Diagram [the Edges]

Control–flow **edges vs.** Object–flow **edges** | Guards



Activity Diagram [the Edges]

Control–flow **edges vs.** Object–flow **edges** | Guards

the **Token Flow** Concept

What is a Token?

It explains how control and data move through the diagram. Think of it like a token or marble traveling along paths to represent the progress of actions or decisions.

A token represents either:

- Control token – showing that control (execution) can move forward.
- Object (data) token – showing that some data or object is available for use in the next action.

These tokens "flow" along the edges (arrows) from one node (activity) to another, controlling execution order.

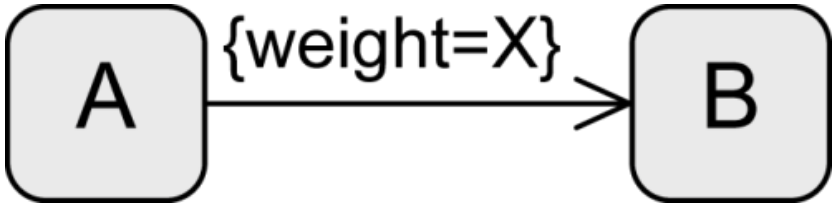
Node Type	**Purpose**	**Token Behavior**
Initial Node	Starts the process	Sends out a control token
Action Node	Performs a task	Waits for token(s) to begin execution
Decision Node	Chooses a path based on conditions	Token flows one of many paths

Activity Diagram [the Edges]

Control–flow **edges vs.** Object–flow **edges** | Guards

the **Token Flow** Concept: the **Weight of Edges**

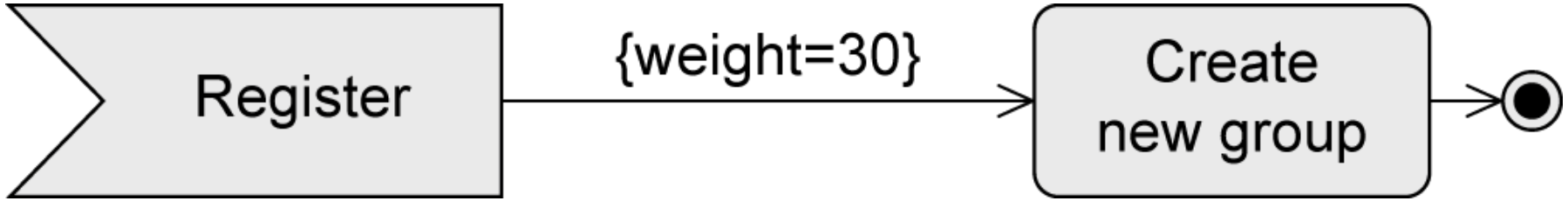
- The weight of an edge is the **Minimal number of tokens that must be present for an action to be executed**.
  - The weight of an edge is specified in curly brackets with the keyword **`weight`**.
  - Default: **`1`**



Activity Diagram [the Edges]

Control–flow **edges vs.** Object–flow **edges** | Guards

the **Token Flow** Concept: the **Weight of Edges**



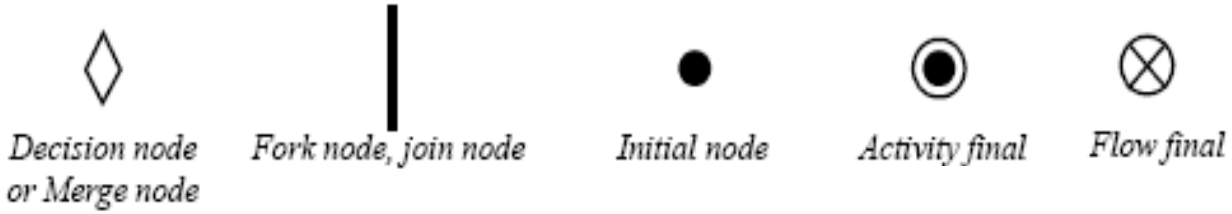
If the signal **`Register`** is received 30 times, meaning that at least 30 students have registered and thus 30 tokens are offered to the subsequent action, then this subsequent action is executed, consumes 30 tokens, and a new group is created.

Activity nodes

Three type of activity nodes:

- **Action nodes**: executable activity nodes; the execution of an action represents some transformations or processes in the modeled system (already seen)

- **Control nodes**: coordinate flows in an activity diagram between other nodes



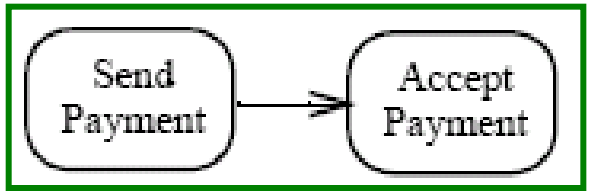
- **Object nodes**: indicate an instance of a particular object, may be available at a particular point in the activity (i.e Pins are object nodes)

Action Node

The fundamental unit of executable functionality in an activity

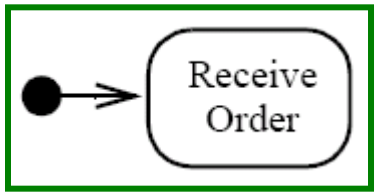
The execution of an action represents some transformations or processes in the modeled system (creating objects, setting attribute values, linking objects together, invoking user-defined behaviours, etc.)

Description of the action behaviour

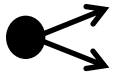


Control nodes – initial nodes

In an *activity* the flow starts in initial nodes, that return the control immediately along their outgoing edges



If an initial node has more than one outgoing edge, only one of these edges will receive control, because initial nodes cannot duplicate tokens



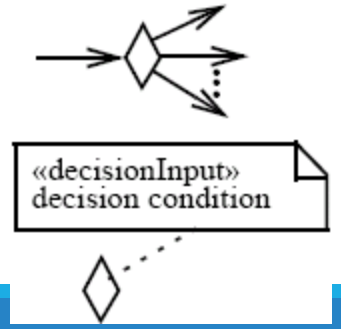
A or B ?

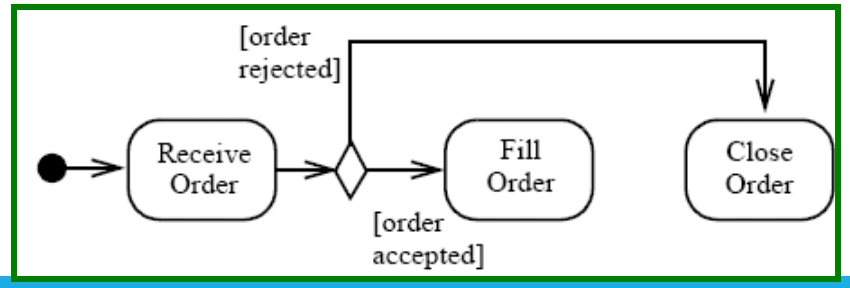
Control nodes – decision nodes

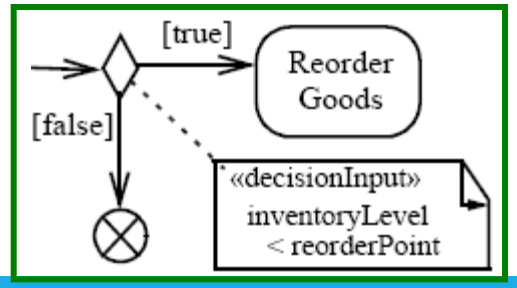
Guards are specified on the outgoing edges or with the stereotype «decisionInput»

There is also the predefined guard **[else]**, chosen only if the token is not accepted by all the other edges

If all the guards fail, the token remains at the source object node until one of the guards accept it





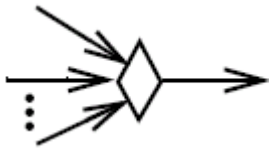


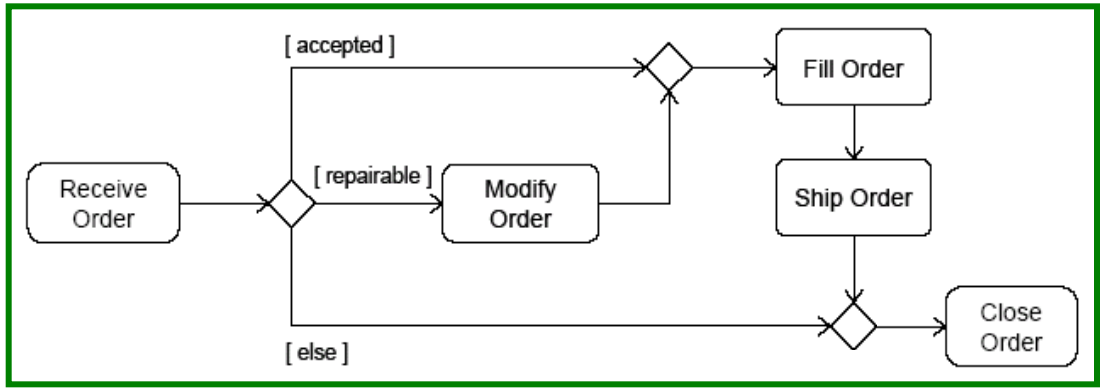
Control nodes – merge nodes

Bring together multiple alternate flows

All controls and data arriving at a merge node are immediately passed to the outgoing edge

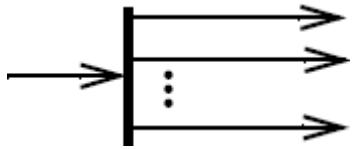
There is no synchronization of flows or joining of tokens

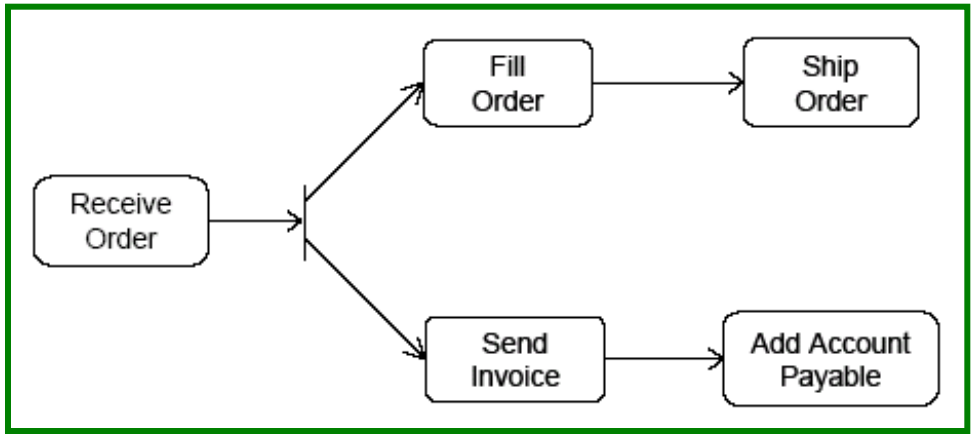




Control nodes – fork nodes

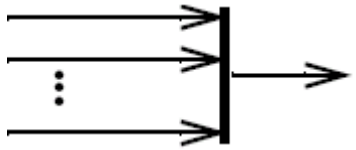
Fork nodes split flows into multiple concurrent flows

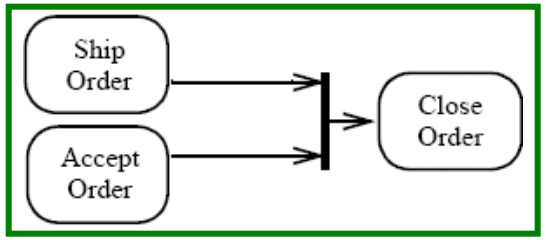




Control nodes – join nodes

Join nodes synchronize multiple flows

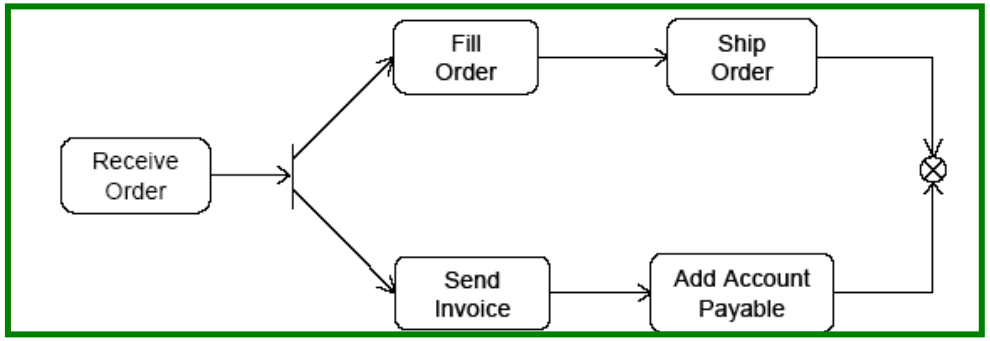




Control nodes – final nodes

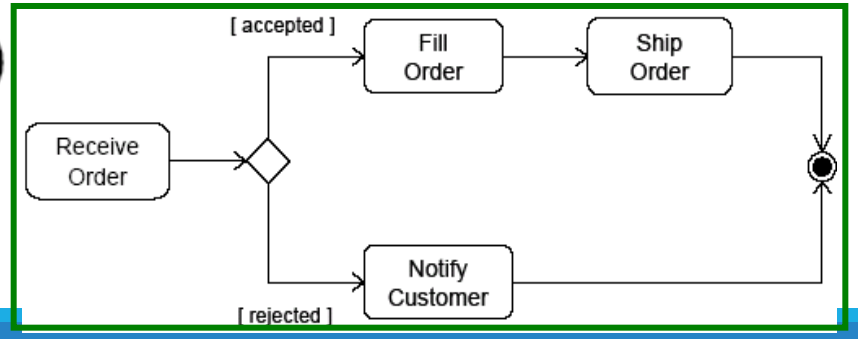
Flow final:

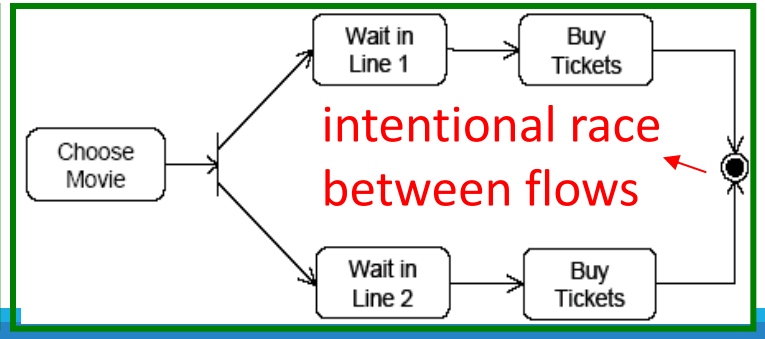
- A Flow Final Node is a special type of final node in UML Activity Diagrams that destroys tokens without stopping the entire activity. It only terminates the flow reaching it, allowing other parts of the activity to continue executing.



Final node:

- the activity is terminated when the first token arrives



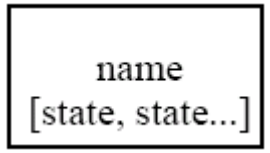


Object nodes

Hold data temporarily while they wait to move through the graph

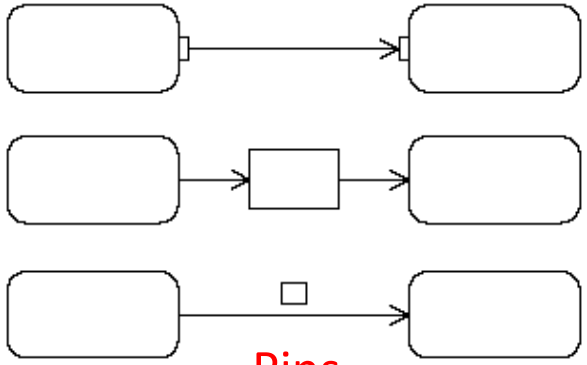
Specify the type of values they can hold (if no type is specified, they can hold values of any type)

Can also specify the state of the held objects

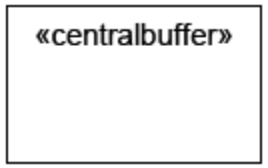


There are four kinds of object nodes:

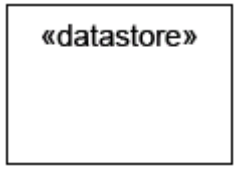
Activity Parameter Nodes



Pins
(three differents notations)



Central Buffer Nodes



Data Store Nodes

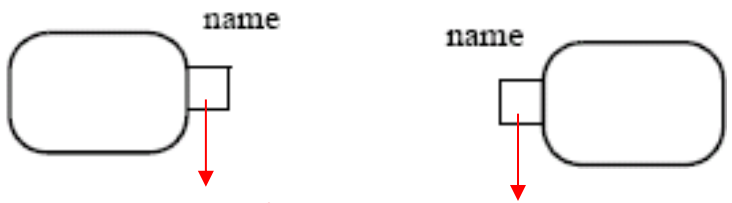
Object nodes – Pins

What are Pins?

Pins are used in **UML Activity Diagrams** to represent **inputs and outputs** for actions. They act as **holders** for data or objects, ensuring that:

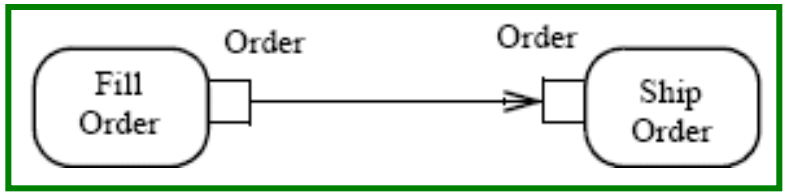
1. Inputs are held until the action starts.
2. Outputs are held until the values move to the next activity.

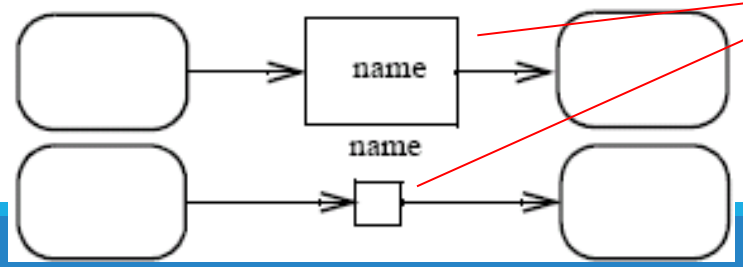
The name of a pin is not restricted: generally recalls the type of objects or data that flow through the pin



Output pins

Input pins





Standalone pin notations: the output pin and the input pin have the same name and same type

Object nodes – centralBuffer

«centralBuffer»
CB

the Central Buffer «centralBuffer»

- For saving and passing on object tokens.
- Transient memory.
- Accepts incoming object tokens from object nodes and passes them on to other object nodes.
- When an object token is read from the central buffer, it is deleted from the central buffer and cannot be consumed again.

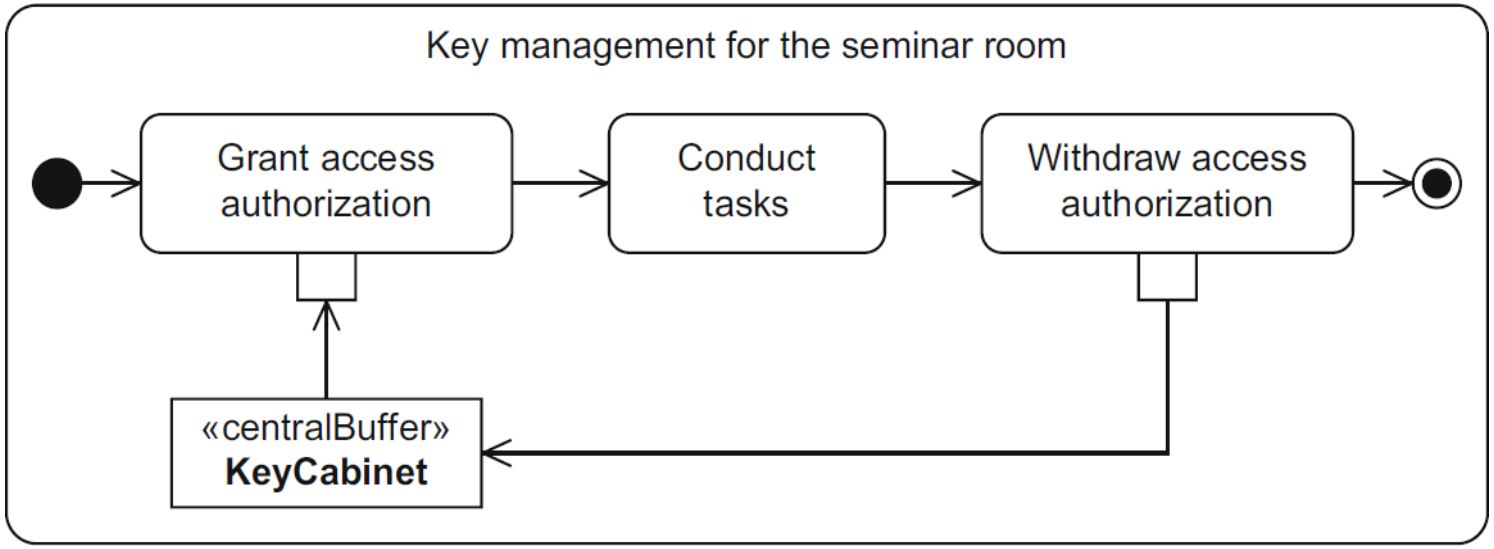
Object nodes – centralBuffer

the Central Buffer «centralBuffer»

"Key management for the seminar room"

This is an **Activity Diagram** that shows how a key is managed (given, used, and returned).

It starts with granting access, followed by conducting tasks, and ends with withdrawing access. The KeyCabinet acts as a central buffer where the key is stored, taken when access is granted, and returned when access is withdrawn.



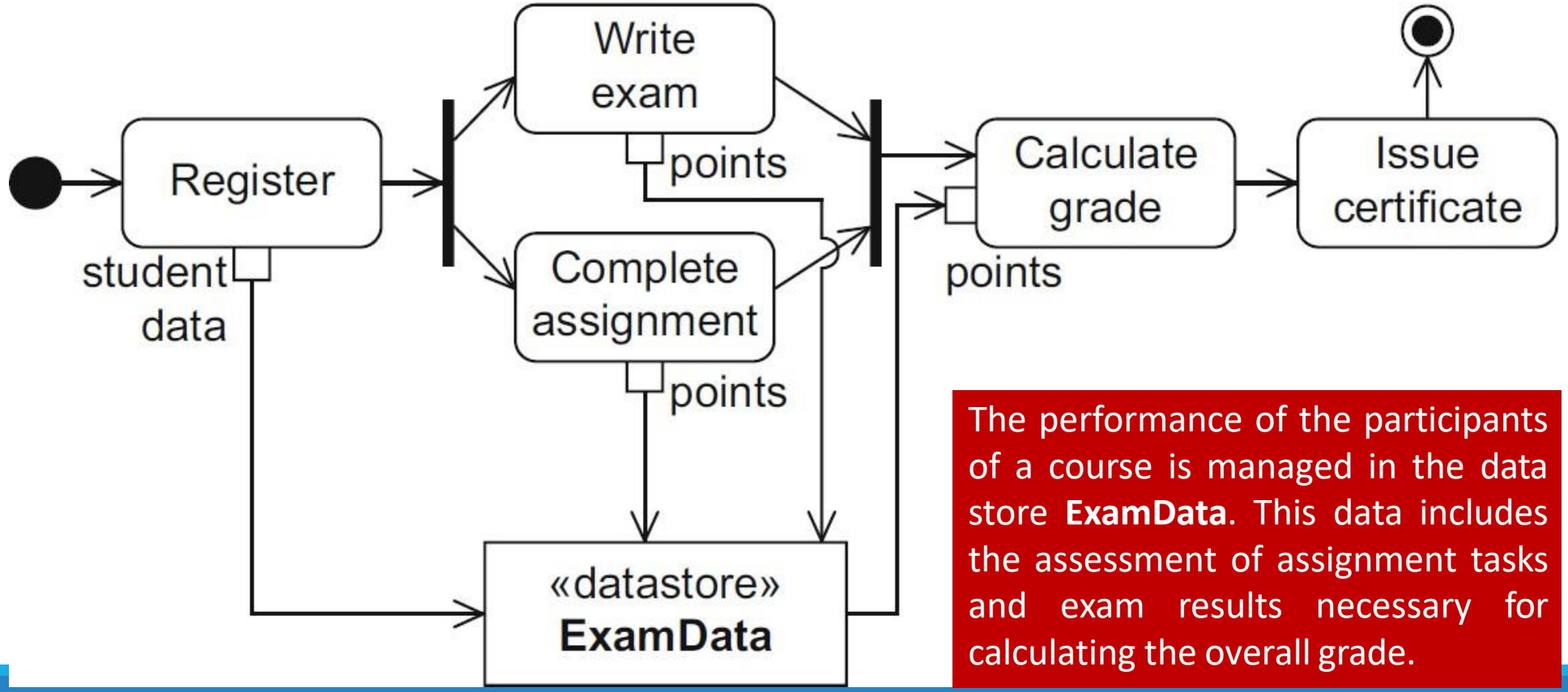
Object nodes – datastore

the Data Store «datastore»

- For saving and passing on object tokens.
- Permanent memory.
- Saves object tokens permanently, passes copies to other nodes.

Object nodes – datastore

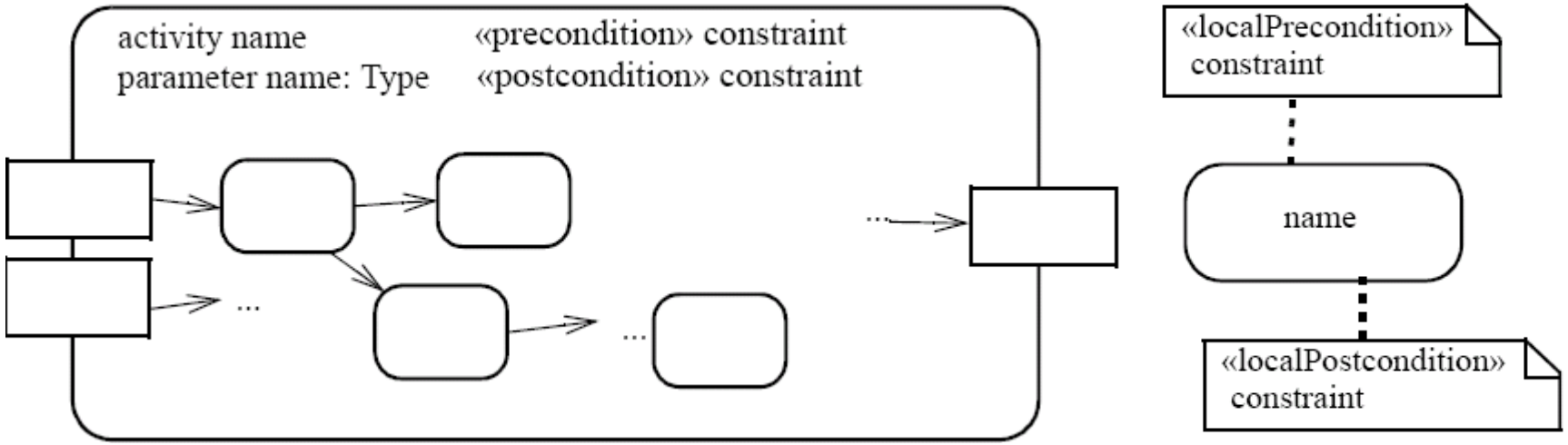
↳ the **Data Store «datastore»**



The performance of the participants of a course is managed in the data store **ExamData**. This data includes the assessment of assignment tasks and exam results necessary for calculating the overall grade.

Pre & post condition

Can be referred to an activity or to an action (local condition)

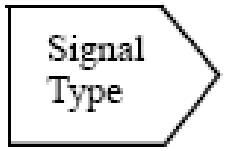


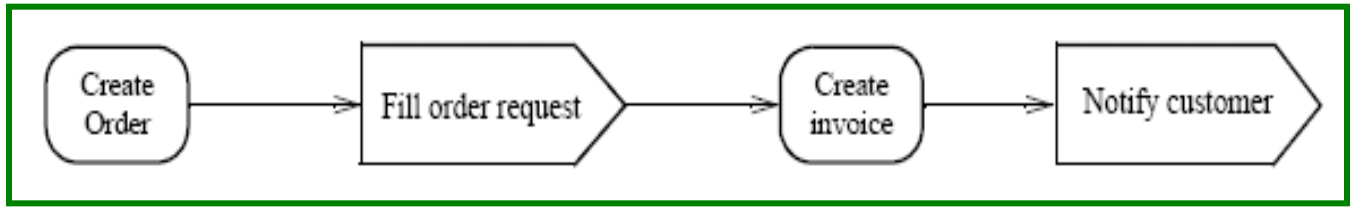
UML intentionally does not specify when or whether pre/post conditions are tested (design time, runtime, etc.)

SendSignalAction

SendSignalAction is an action in a UML **Activity Diagram** that is used to send a **signal** (event) to other parts of the system.

It represents an asynchronous message or event that is triggered without waiting for a response.



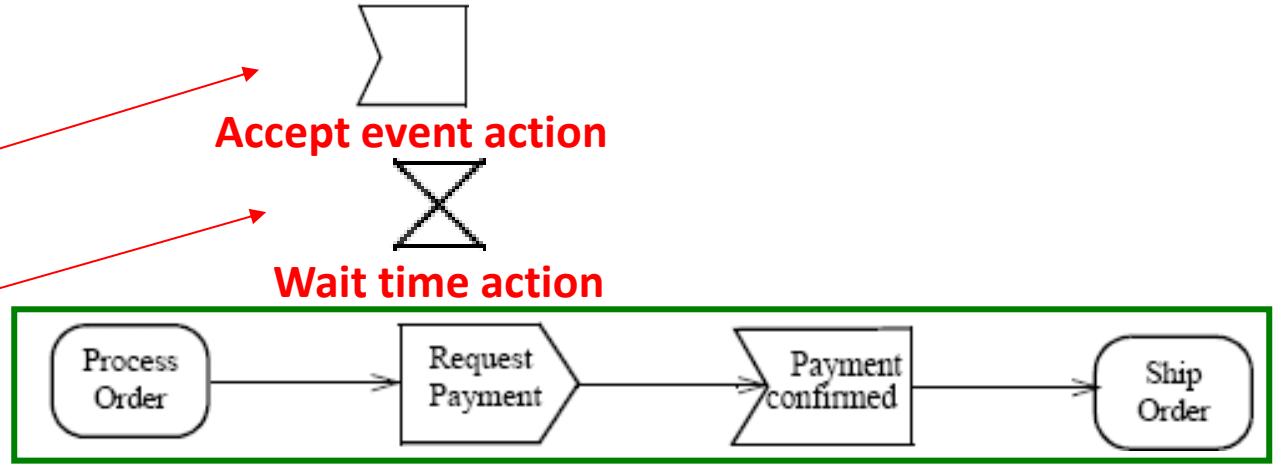


AcceptEventAction

AcceptEventAction is an action that waits for an **event to occur**. It is used when an activity needs to **pause and wait** for an event, such as receiving a message, user input, or a signal.

Two kinds of AcceptEventAction:

- **Accept event action** – accepts signal events generated by a SendSignalAction
- **Wait time action** – accepts time events

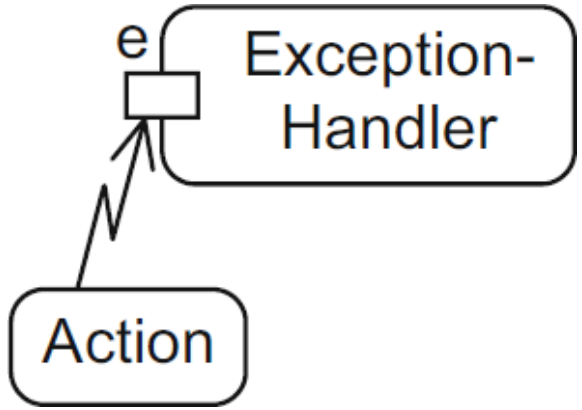


Activity Diagram [Exception Handling]

the Exception Handler

Predefined exceptions; *If an action has an exception handler for a specific error situation, this exception handler is activated when an exception occurs.*

- Thus, it defines how the system should react in a specific error situation *(minimizing the effects of this error)*.
- The exception handler replaces the action where the error occurred.

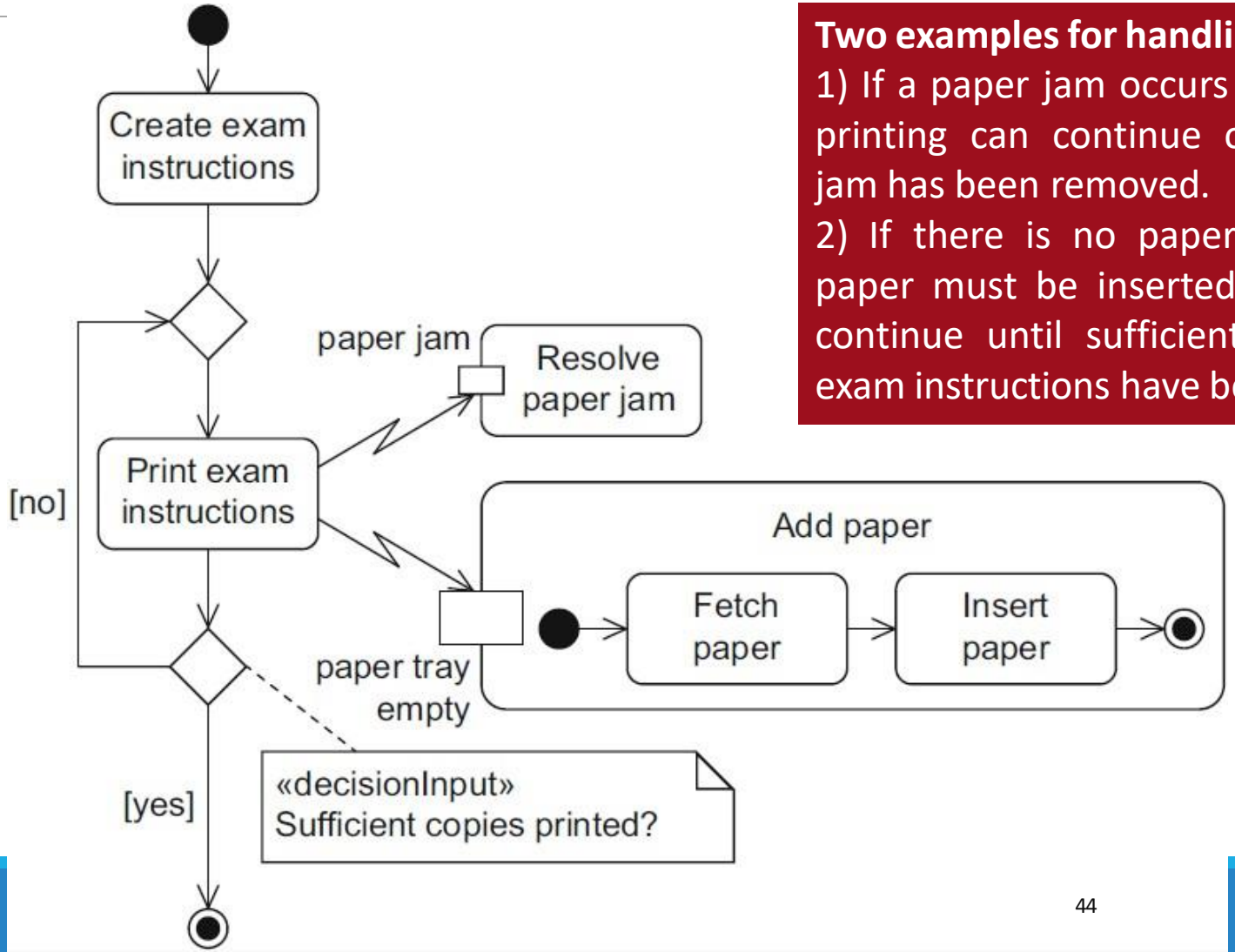


If the error **e** occurs :

- All tokens in **Action** are deleted.
- The exception handler is activated.
- The exception handler is executed instead of **Action**.

Activity Diagram [Exception Handling]

the Exception Handler



Two examples for handling exceptions:

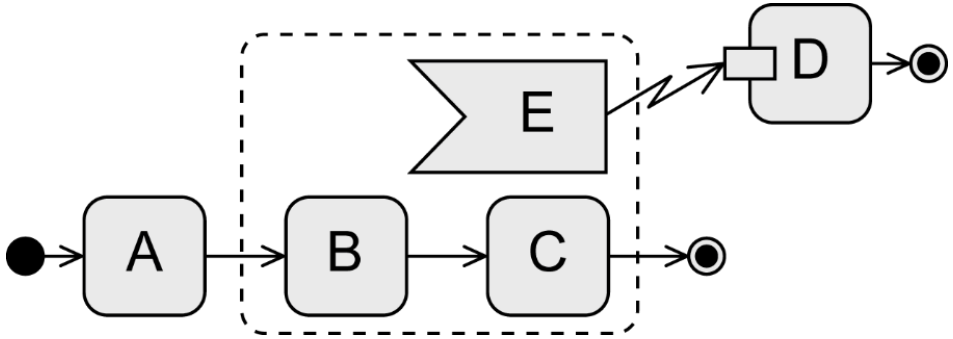
1) If a paper jam occurs during printing, printing can continue once the paper jam has been removed.

2) If there is no paper in the printer, paper must be inserted for printing to continue until sufficient copies of the exam instructions have been printed.

Activity Diagram [Exception Handling]

the Interruptible Activity Region

Defining a group of actions whose execution is to be terminated immediately if a specific event occurs. In that case, some other behavior is executed.



If **E** occurs while **B** or **C** are executed :

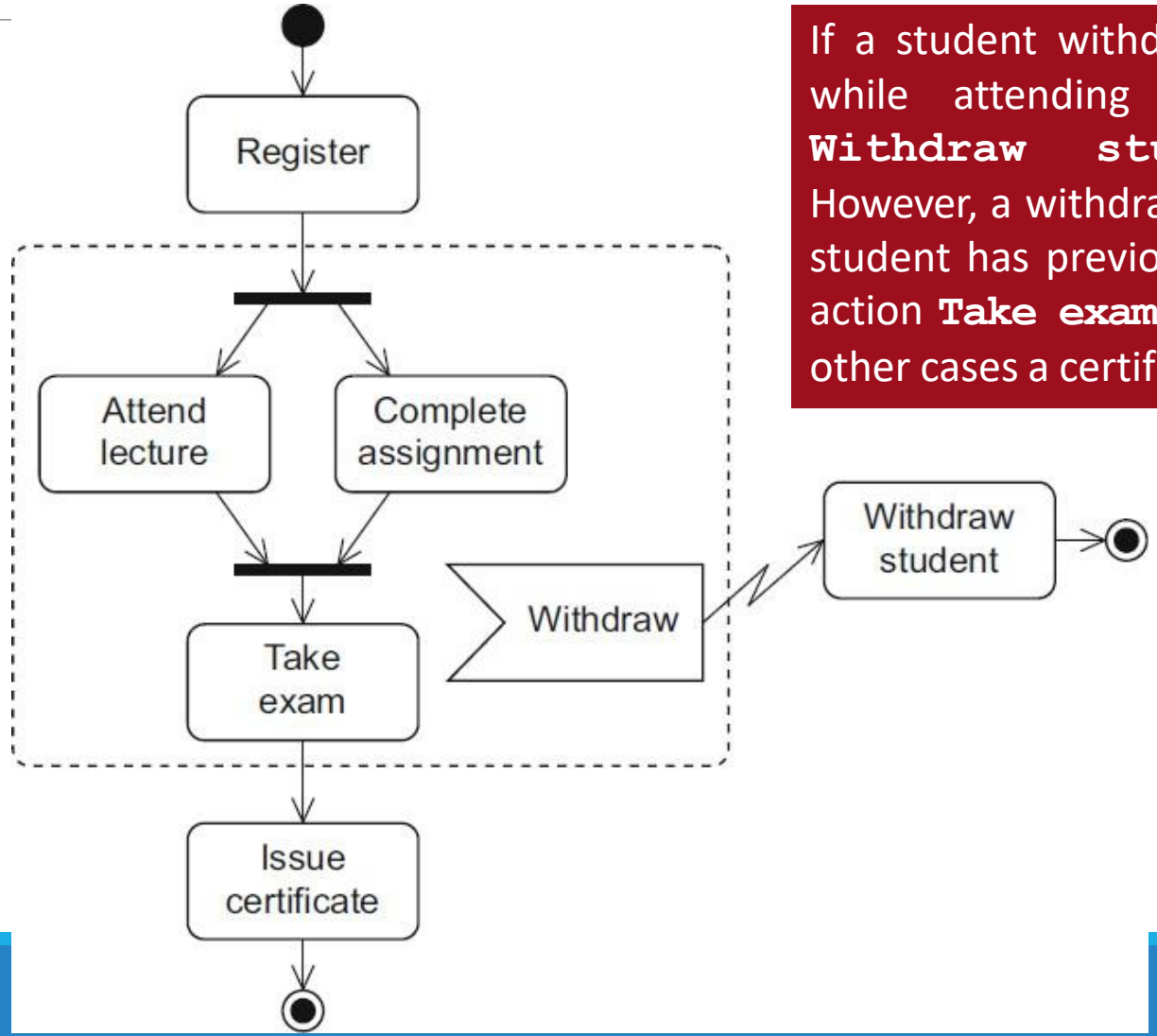
- Exception handling is activated.
- All control tokens within the dashed rectangle (= within **B** and **C**) are deleted.
- **D** is activated and executed.

No “jumping back” to the regular execution!

an **Example**

Activity Diagram [Exception Handling]

the Interruptible Activity Region



If a student withdraws from the university while attending a course, the action **`Withdraw student`** is executed. However, a withdrawal is only possible if the student has previously registered and if the action **`Take exam`** has not yet ended. In all other cases a certificate is issued.

Activity Diagram [an example]

the Process of Issuing a Student ID

The process of issuing a student ID can be described as follows:

1. To obtain a student ID, the student must request this ID from an employee of the student office.
2. The employee hands the student the forms that the student has to fill out to register at the university.
3. These forms include the student ID itself, which is a small, old-style cardboard card.
4. The student has to enter personal data on this card and the employee confirms it with a stamp after checking it against certain documents.
5. The student ID is only valid if it has a current semester label.
6. Once the student has filled out the forms, the student returns them to the employee in the student office and hands over documents such as photo identification, school-leaving certificate, and birth certificate.

Activity Diagram [an example]

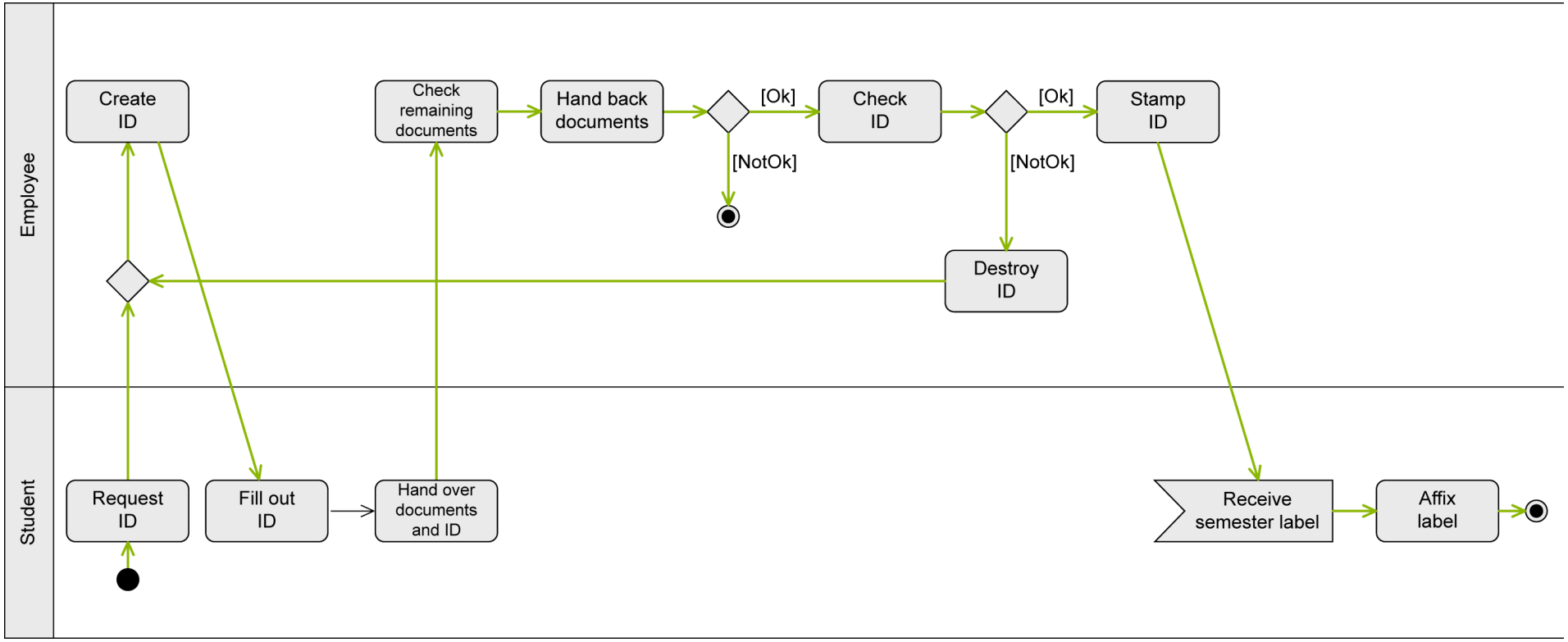
the Process of Issuing a Student ID

The process of issuing a student ID can be described as follows:

7. The employee checks the documents. If the documents are incomplete or the student is not authorized to receive a student ID for the university, the process is terminated immediately.
8. If the documents are all in order, the employee checks whether the student has filled out the student ID correctly.
9. If there are any errors, this ID is destroyed and the student has to fill out another one.
10. Otherwise the ID is stamped.
11. However, the student ID is not valid until it bears the semester label sent to the student by post.

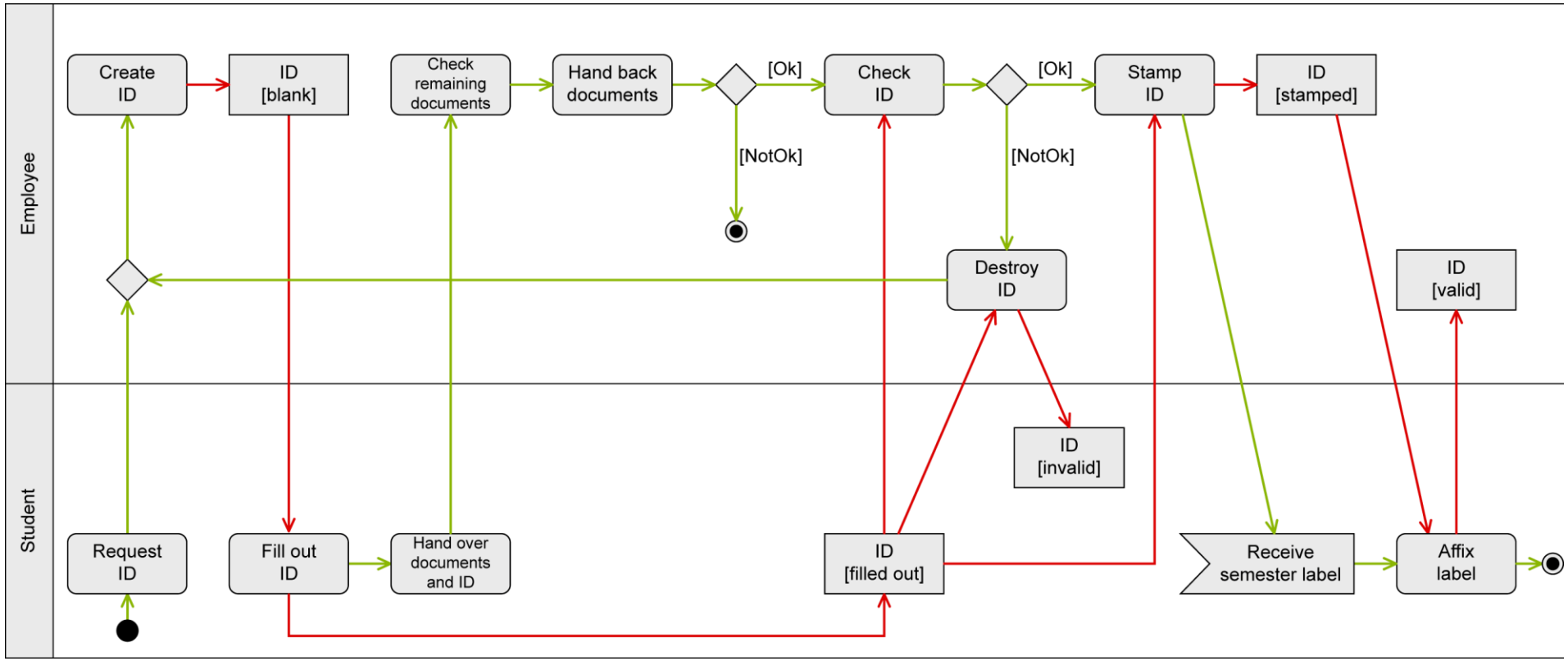
Activity Diagram [an example]
the Process of Issuing a Student ID

Modelling only the Control Flow :



Activity Diagram [an example]
the Process of Issuing a Student ID

Modelling the Object Flow *(changes to the student ID are shown)* **:**



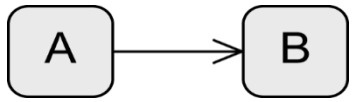
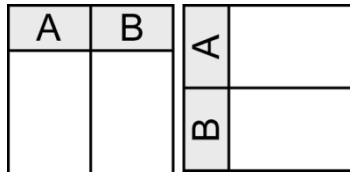
.. Control flow (green) and object flow (red).

Activity Diagram

a Summary of Notation Elements

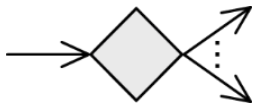
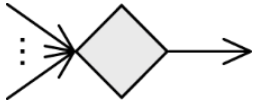
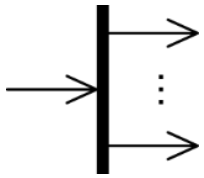
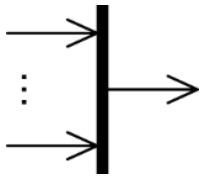
Name	Notation	Description
Action Node	Action	Represents an action (i.e., atomic!).
Activity Node	Activity	Represents an activity (can be broken down further).
Initial Node	●	Start of the execution of an activity.
Activity–Final Node	◉	End of ALL execution paths of an activity.
Flow–Final Node	⊗	End of ONE execution path of an Activity.

Activity Diagram
a Summary of Notation Elements

Name	Notation	Description
Edge		Connection between the nodes of an activity.
Partition (Lanes / Swimlanes)		Grouping of nodes and edges within an activity.

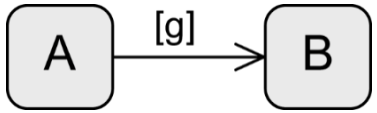
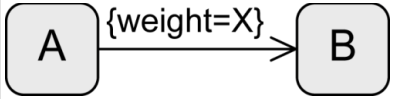
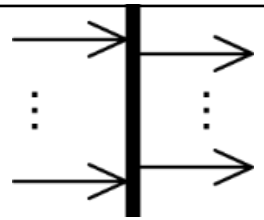
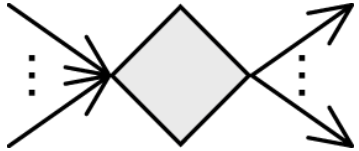
Activity Diagram

a Summary of Notation Elements

Name	Notation	Description
Decision Node		Splitting of one execution path into alternative execution paths.
Merge Node		Merging of alternative execution paths into one execution path.
Parallelization Node *(Fork Synchronization Bar)*		Splitting of one execution path into concurrent execution paths.
Synchronization Node *(Join Synchronization Bar)*		Merging of concurrent execution paths into one execution path.

Activity Diagram

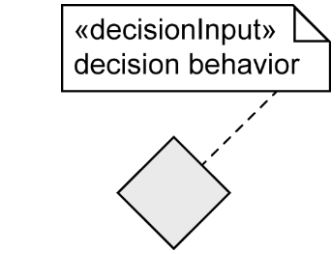
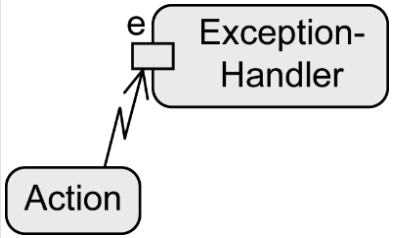
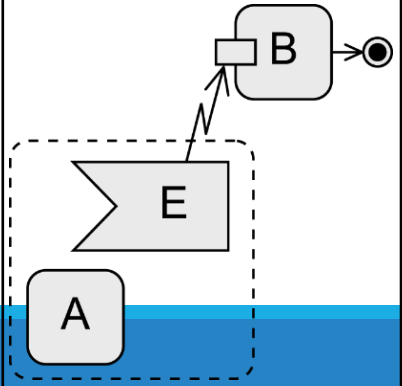
a Summary of Notation Elements

Name	Notation	Description
Guard [*condition*]		Only if guard "g" evaluates to true, the control/object flow continues.
Weight of an Edge		Minimal number of tokens "X" that must be present for action B to be executed.
Combined Parallelization & Synchronization Node		A bar with multiple incoming edges & multiple outgoing edges.
Combined Decision & Merge Node		A node with multiple incoming edges & multiple outgoing edges.

Activity Diagram
a Summary of Notation Elements

Name	Notation	Description
Send–Signal Action	S	Transmission of a signal `S` to a receiver.
Asynchronous Accept (Timing) Event Action	E or T	Wait for an event `E` or a time event `T`.
Object Node	Object	Contains data or objects.
Parameter for Activities Parameter for Actions (Pins)	Activity Action	Contains data and objects as input and output parameters.

Activity Diagram
a Summary of Notation Elements

Name	Notation	Description
Decision Behavior		The behavior necessary for the evaluation of the guards.
Exception Handler		Exception handler is executed instead of the action in the event of an error e.
Interruptible Activity Region		Flow continues a different path if event E is detected.

Activity Diagram
a Summary of Notation Elements

Name	Notation	Description
Special Object Nodes: Central Buffer «centralBuffer»	«centralBuffer» **CB**	Represents transient memory; the information can only be used once *(once it has been read from the central buffer and forwarded it is lost).*
Special Object Nodes: Data Store «datastore»	«datastore» **DS**	Represents permanent memory; the information can be used as often as required, provided it has been saved once in the data store.

Outline of course

Contents going to be covered during course:

- Course Outline, Introduction to Software Engineering
- Software Processes & Development Life-Cycles
- Requirements Engineering
- Use-Case Diagram
- Activity Diagram
- UML Interaction Diagrams (Sequence, Collaboration, & System Sequence Diagrams)
- UML Static Diagrams (Class, Object)
- Introduction to Design Patterns
- A Brief Introduction to Testing

Any Questions